



CSS

Web programming I



CSS

Student is able to

design and implement interactive and responsive web pages using HTML, CSS, JavaScript, and Bootstrap.



CONTENT

CSS basics

- What is CSS and its role?

- How to apply CSS rules to HTML elements?

THE CASCADE

- What does "cascading" mean in CSS, and why does it matter?

- How do conflicting CSS rules get resolved?

STYLING CONTENT

- How to style text content?

- How to add backgrounds and borders to elements?

LAYOUT

- How to control where elements appear on the page?

- How to make layouts responsive across different screen sizes?



CONTENT

CSS basics

What is CSS and its role?

How to apply CSS rules to HTML elements?

THE CASCADE

What does "cascading" mean in CSS, and why does it matter?

How do conflicting CSS rules get resolved?

STYLING CONTENT

How to style text content?

How to add backgrounds and borders to elements?

LAYOUT

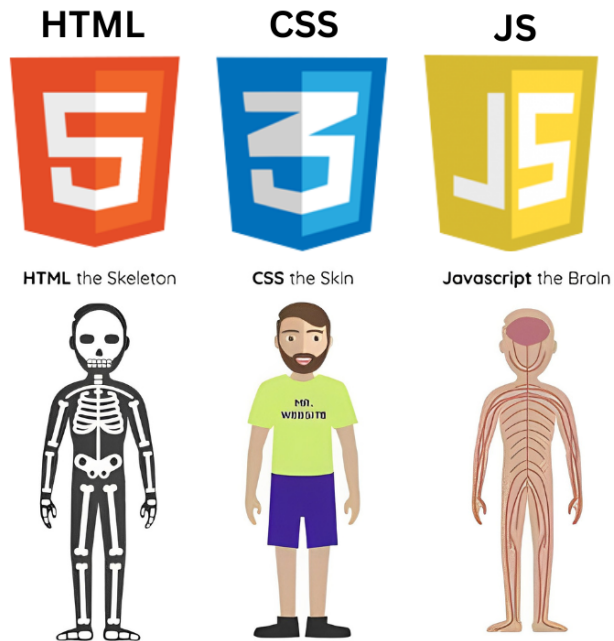
How to control where elements appear on the page?

How to make layouts responsive across different screen sizes?



CSS basics › Role of CSS in client technologies

Role of CSS in client technologies





HTML should only represent the **content**

News

- [Recycling of electric vehicle batteries needs to be improved in Europe](#)
8.6.2026
- [Innovation Pathways Podcast series: Entrepreneurial culture as a driver of innovation](#)
5.6.2026
- [Doctoral defence of Hamza Rehman, MSc, 12.6.2026: Nanocarbon-based terahertz components](#)
5.6.2026

Events

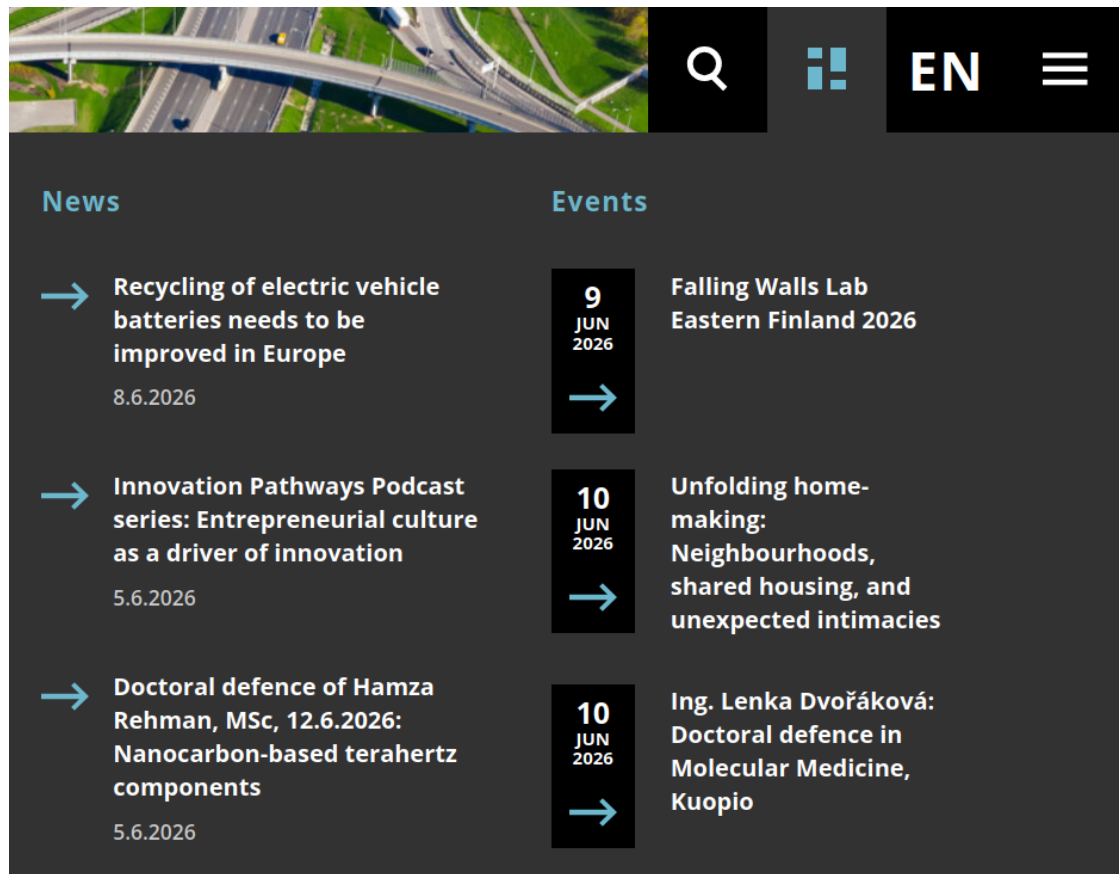
- 9
Jun
2026
[Falling Walls Lab Eastern Finland 2026](#)
- 10
Jun
2026
[Unfolding home-making: Neighbourhoods, shared housing, and unexpected intimacies](#)
- 10
Jun
2026
[Ing. Lenka Dvořáková: Doctoral defence in Molecular Medicine, Kuopio](#)

[Find more news and events](#)

Vaihda kieltä, nykyinen kieli: suomi Switch language, current language: english FI EN

- [In English](#)
- [Suomeksi](#)

Language called
CSS is used
to create rules
about **style**





CSS syntax

selector **body {**
property **color: red;**
value



Property-value examples

background-color: lightblue;

text-align: center;

cursor: pointer;

background-image: url("exampleImageName.jpg");

display: none;



Adding CSS styling

inline

style attribute to the HTML element

Only applies to that tag, not shared with other tags.

Style properties are written as "name: value" separated by semicolons

```
<p style="color: yellow; font-style: italic;">text</p>
```

internal

<style> block in the <head> of the document

The style is applied to all elements on the page.

```
<head>
  <style type="text/css">
    b { font-size: 12pt; color: green; }
    /* Headers */
    h1, h2 { margin-left: 15pt; color: red; }
  </style>
</head>
```

external

separate file with the style rules

This completely separates content and presentation.

Can be used in multiple HTML files.

```
<link rel="stylesheet" type="text/css" href="style.css">
```



Default CSS values

- each HTML tag has default CSS values, which may vary depending on the browser
- defining our own personal styles override the default rules

Level 1 heading

This paragraph contains text marked as *important* and other text marked as **very important**.

This is the second paragraph.

```
<!doctype html>
<html>
  <head>
    <meta charset="utf-8">
    <title> Text marks </title> </head>
  <body>
    <h1>Level 1 heading</h1>
    <p> This paragraph contains text marked
      as <em>important</em> and other text
      marked as <strong>very important.
      </strong></p>
    <p> This is the second paragraph.</p>
  </body>
</html>
```



Targeting elements by tag or attribute

```
<p class="myClass" id="myId">Hello</p>
```

Selectors can target elements using

- type selector (by tag name)
- class selector (by class attribute)
- ID selector (by id attribute)
- + combination of tag and attribute

```
p { color: red; }
```

```
.myClass { color: red; }
```

```
#myId { color: red; }
```

```
p.myClass { color: red; }
```

Remember: ids must be unique on the page, but a class can be used on multiple elements.



Targeting elements by relationships and states

Combination selectors (relationships between elements):

- multiple selectors, e.g. both `<h1>` and `<h2>`
- descendant (any depth), e.g. all `<li>` inside `<nav>` at any level
- child (direct parent), e.g. only `<li>` directly inside `<ul>`
- adjacent sibling, e.g. `<p>` immediately after `<h1>`

```
h1, h2 { properties }
```

```
nav li { properties }
```

```
ul > li { properties }
```

```
h1 + p { properties }
```

Pseudo-class (element state):

- e.g. `:hover`, `:focus`, `:visited`, `:first-child`
- applies when element is in a specific state, e.g. `<a>` when the mouse is over it

```
a:hover { properties }
```

Universal selector `*`, define a style for all tags

```
* { properties }
```



CONTENT

CSS basics

What is CSS and its role?

How to apply CSS rules to HTML elements?

THE CASCADE

What does "cascading" mean in CSS, and why does it matter?

How do conflicting CSS rules get resolved?

STYLING CONTENT

How to style text content?

How to add backgrounds and borders to elements?

LAYOUT

How to control where elements appear on the page?

How to make layouts responsive across different screen sizes?



What is cascading?

Cascade refers to something that flows or occurs in a series of stages, originally a sequence of waterfalls

A cascade is a chain of events or a flowing sequence.

Cascading in Cascading Style Sheet = which style rule apply when multiple rules target the same element

- multiple style declarations “flow down” from different sources, and is resolved into a single result





Three levels of cascading

1. Origin and importance

`!important` > inline > internal > external > browser default

Styles marked with `!important` gets the highest priority

2. Specificity - hierarchy of CSS selectors

3. Source order - order in which a CSS rules are defined

the rule defined later takes the priority



Specificity Hierarchy

inline > id > class > element

Every CSS selector has its place in the specificity hierarchy.

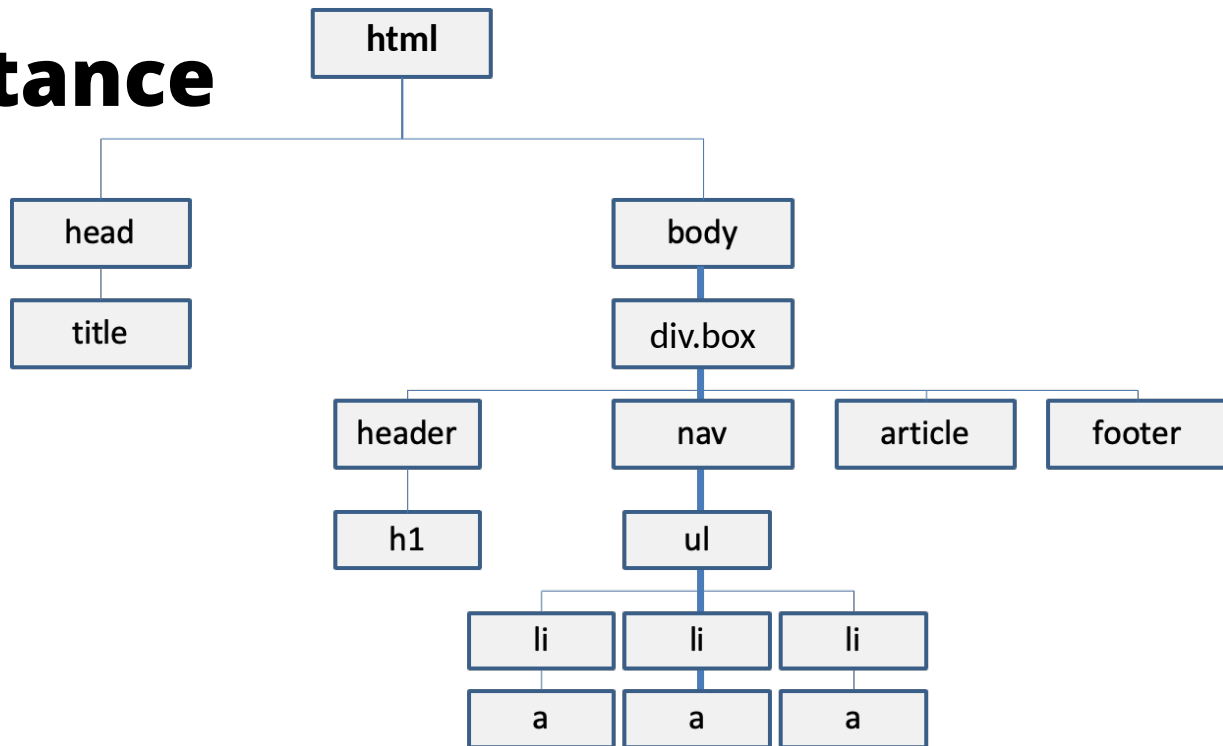
1. Inline styles – e.g.: `<h1 style="color: pink;">`
2. IDs – e.g. : `#navbar`
3. Classes, pseudo-classes, attribute selectors – e.g. : `.test`, `:hover`, `[href]`
4. Elements and pseudo-elements – e.g. : `h1`, `::before`

There's numerical value for specificity, each ID value, class value etc. are calculated



CSS inheritance

Rules are inherited from parent elements to children unless they are overridden by a more specific rule on the child.



```
body { color: blue; } /* all text becomes blue */  
h1 { color: red; } /* but h1 elements override this */
```



CONTENT

CSS basics

What is CSS and its role?

How do we apply CSS rules to HTML elements?

THE CASCADE

What does "cascading" mean in CSS, and why does it matter?

How do conflicting CSS rules get resolved?

STYLING CONTENT

How to style text content?

How to add backgrounds and borders to elements?

LAYOUT

How to control where elements appear on the page?

How to make layouts responsive across different screen sizes?



Text properties

`text-align: center | left | right | justify`

`text-decoration` is a shorthand that combines:

- `text-decoration-line: overline | line-through | underline | none`
- `text-decoration-color: color`
- `text-decoration-style: solid | dashed | dotted | wavy | ...`

For example `text-decoration: none; /* remove underline from links */`

/ With individual properties */*

```
.discount {  
  text-decoration-line: line-through;  
  text-decoration-color: red;  
  text-decoration-style: wavy;  
}
```

/ Same result with shorthand */*

```
.discount { text-decoration: line-through red wavy; }
```



Styling the content › Font properties

Font properties

1em is equal to the current font size. The default text size in browsers is 16px. So, the default size of 1em is 16px.

```
P { font-size: 40px; }
```

```
/* 40px/16=2.5em */  
P { font-size: 2.5em; }
```

```
P { font-size: 25vw; }
```

This is 40px

This is 2.5em

This is 25vw

```
.one {  
  font-family: "Times New Roman",  
  Times, serif;  
}
```

```
.two {  
  font-family: Arial, Helvetica, sans-  
  serif;  
}
```

```
.three {  
  font-family: "Lucida Console",  
  "Courier New", monospace;  
}
```

This is text with a serif font

This is text with a sans-serif font

This is text in monospace



Google Fonts

1. Go to fonts.google.com and choose a font
2. Click "Get font" → "Get embed code"
3. Add to <head> in index.html:

```
<link rel="preconnect" href="https://fonts.googleapis.com">  
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>  
<link rel="stylesheet" href="https://fonts.googleapis.com/css2?family=Sofia:wght@400;600;700&display=swap">
```

4. Use in CSS:

```
body {  
  font-family: 'Sofia', sans-serif;  
}
```

Sofia Font



Background

```
Body {background-image: url("photo.jpg"); }
```

By default, a background image repeats to fill the area (background-repeat: repeat) and extends to the border of the element (background-clip: border-box)

You can change both

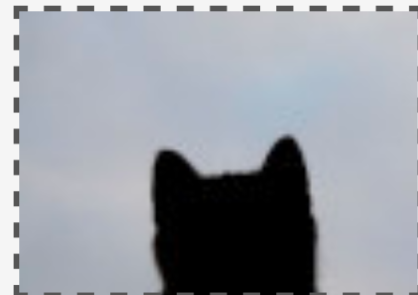
background-clip controls where the image is drawn:

- **border-box** (default): image extends to the border
- **padding-box**: image stops at the padding
- **content-box**: image only behind the content

background-repeat controls how the area is filled:

- **repeat** (default): tiles the image
- **no-repeat**: shows the image once
- **repeat-x/repeat-y**: tiles only horizontally / vertically

default



background-clip: content-box



```
.catbox {  
  width: 300px;  
  height: 200px;  
  padding: 20px;  
  border: 5px dashed #555;  
  margin-bottom: 30px;  
  background-image: url("cat.jpg");  
  background-repeat: no-repeat;  
  background-size: cover;  
}  
.catcontent {  
  background-clip: content-box;  
}
```

```
<div class="catbox"></div>  
<div class="catbox catcontent"></div>
```



Styling the content >

Background positioning and sizing

Once you have a background image, you can control how it fits:

background-repeat:

- no-repeat | repeat | repeat-x | repeat-y

background-size:

- cover: fills the area, may crop
- contain: fits inside the area, may leave space
- 100% 100%: stretch to fit

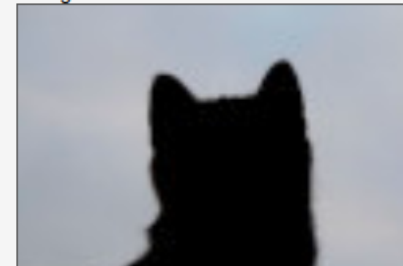
background-position:

- center | top | bottom | left | right

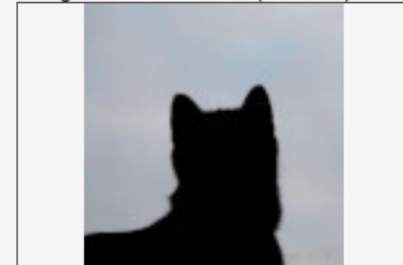
```
.box {  
  border: 2px solid #555;  
  background-image: url("cat.jpg");  
}  
.cover {  
  background-size: cover;  
  background-position: center;  
  background-repeat: no-repeat;  
}  
.contain {  
  background-size: contain;  
  background-position: center;  
  background-repeat: no-repeat;  
}  
.repeat {  
  background-size: 80px 80px;  
  background-repeat: repeat;  
}
```

```
<div class="box cover"></div>  
<div class="box contain"></div>  
<div class="box repeat"></div>
```

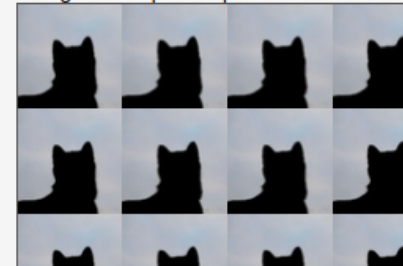
background-size: cover



background-size: contain (centered)



background-repeat: repeat





Border

```
p { border: 2px solid blue; }
```

- border-width: 5px, medium, thick, ...
- border-style: dotted, dashed, solid, double, ...
- border-color: red, blue, #f0f0f0, ...

Each side separately:

```
p {  
  border-bottom: 6px solid red;  
}
```

Some text



Rounded corners with border-radius:

```
p {  
  border: 12px solid red;  
  border-radius: 5px;  
}
```

Roundest border





CONTENT

CSS basics

What is CSS and its role?

How do we apply CSS rules to HTML elements?

THE CASCADE

What does "cascading" mean in CSS, and why does it matter?

How do conflicting CSS rules get resolved?

STYLING CONTENT

How to style text content?

How do add backgrounds and borders to elements?

LAYOUT

How to control where elements appear on the page?

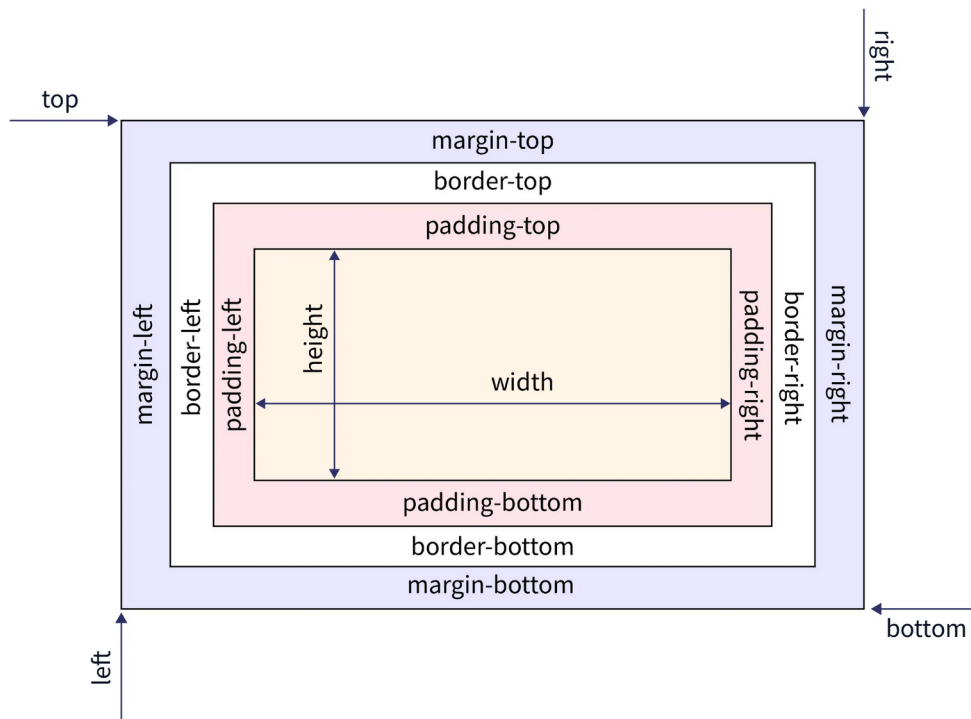
How to make layouts responsive across different screen sizes?



Box model

margins = space around element,
outside of borders

padding = space around element,
inside of borders





Property: margin

```
/* Specify the same value for all sides*/
```

```
div {  
  margin: 100px;  
}
```

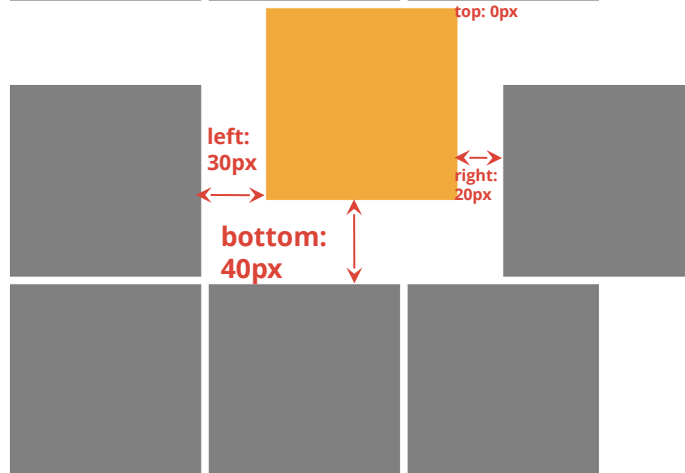
```
/* Specify each side's margin*/
```

```
/* Option 1 */
```

```
div {  
  margin-top: 100px;  
  margin-bottom: 20px;  
  margin-right: 40px;  
  margin-left: 30px;  
}
```

```
/* Option 2 */
```

```
div {  
  margin: 100px 40px 20px 30px;  
}
```



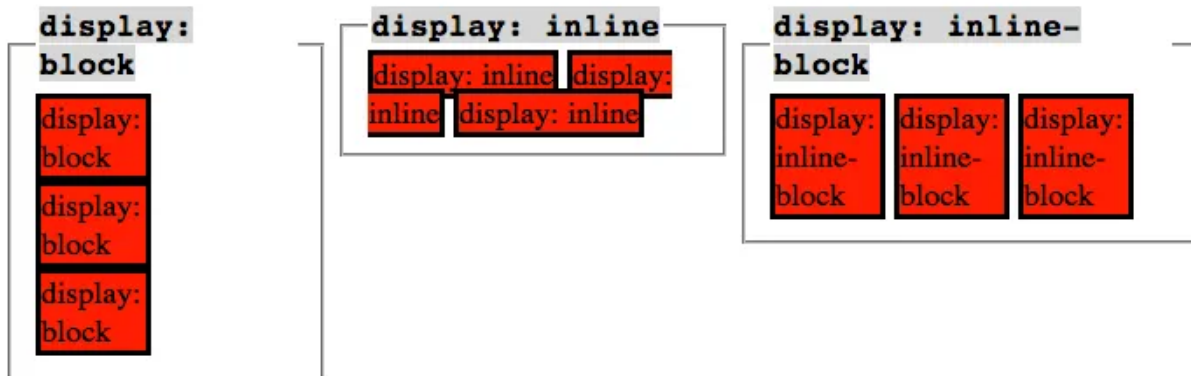
```
div {  
  width: 100px;  
  display: inline-block;  
  height: 100px;  
  margin: 1px;  
}
```

```
#thebox{  
  background-color: orange;  
  margin: 0px 20px 40px 30px;  
}
```



Layout > Property: display

Property: display



display: none element is not displayed

display: inline displays an element as an inline element

display: block displays an element as a block element

display: inline-block displays an element as an inline-level block container (formatted as an inline element, but you can apply height and width values)

display: flex displays an element as a block-level flex container

display: grid displays an element as a block-level grid container



Layout > Property: position

Property: position

```
position: static;
```

```
position: static;
```

```
position: static;
```

```
position: relative;  
top: 100px;
```

```
position: absolute;  
top: 100px;  
right: 100px;
```

- **position: static** (default): normal flow
- **position: relative** positioned relative to its original position
- **position: absolute** positioned relative to ancestor
- **position: fixed** Stays fixed when scrolling

```
<div style="position:static;"></div>
```

```
<div style="position:relative;top: 100px;"></div>
```

```
<div style="position:static;"></div>
```

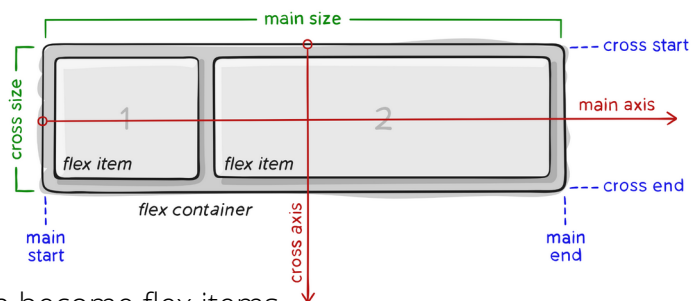
```
<div style="position:absolute; top: 100px; right: 100px;"></div>
```

```
<div style="position:static;"></div>
```



Layout > Flexbox container

Flex container

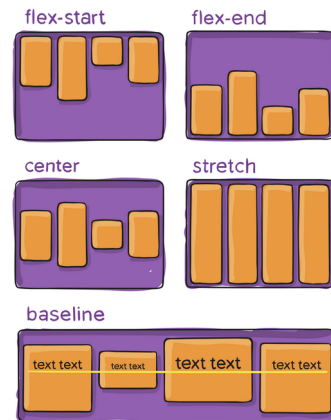
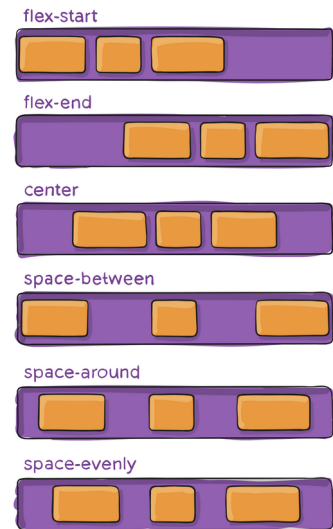


A flex container is an HTML element whose children become flex items. The flex-container rules control how the children are arranged inside.

```
<div class="myflex">  <!-- flex container -->
  <div>1</div>         <!-- flex item -->
  <div>2</div>         <!-- flex item -->
  <div>3</div>         <!-- flex item -->
</div>
```

```
.myflex {
  display: flex; /* ← THIS makes flex container */
  flex-direction: row; /* arrange children horizontally */
  justify-content: center; /* center them on the main axis */
  align-items: center; /* center them on the cross axis */
}
```

flex-direction: row | row-reverse | column | column-reverse;
flex-wrap: nowrap | wrap | wrap-reverse;
justify-content: flex-start | flex-end | center |
space-between | space-around | space-evenly | start | end ...;
align-items: stretch | flex-start | flex-end | center | baseline;





Flex items

Inside a flex container, each item can grow to fill space.

```
/* all items: equal share */
```

```
.item {  
  flex: 1;  
}
```



```
/* this item: twice as much */
```

```
.item2 {  
  flex: 2;  
}
```





Responsive design: media queries

The @media rule allows to define different style rules for different media types.

Media queries can be used to check many things, such as:

- width and height of the viewport
- orientation of the viewport (landscape or portrait)
- Resolution

The media query will only apply if the media type is screen and the viewport is 480px wide or wider.

```
body {  
  background-color: pink;  
}  
  
@media screen and (min-width:  
480px) {  
  body {  
    background-color:  
lightgreen;  
  }  
}
```

Only work correctly if the HTML includes this metadata tag:

```
<meta name="viewport"  
content="width=device-width, initial-  
scale=1.0">
```



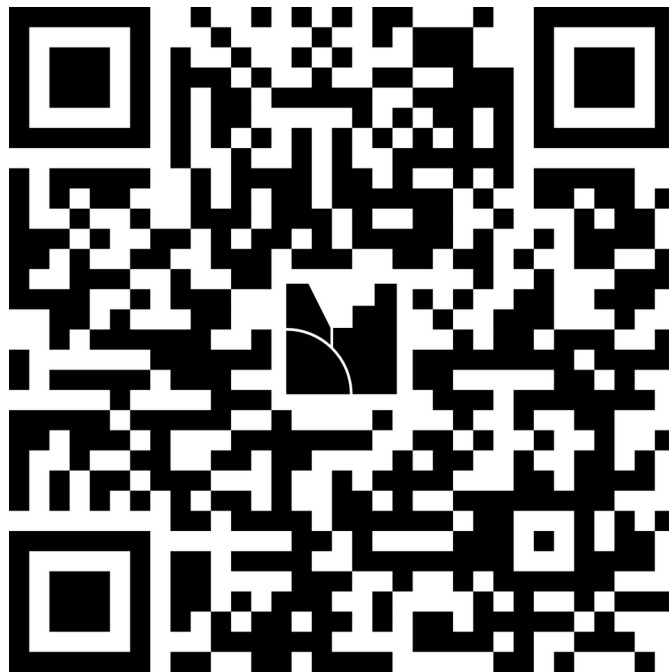
Quick check

Go to

www.menti.com

Enter the code

2402 1274



Or use QR code



Summary

- HTML for content, CSS for the style and layout
- Rules are applied inline, internal, or external; targeting elements with tag, class, id, or combination selectors.
- Cascading = multiple style rules flow down and get resolved into a single result when they target the same element; conflicts are resolved by origin/importance, specificity, and source order.
- Text is styled with properties like text-align, font-size, font-family, and color
- Backgrounds (color or image) and borders define an element's visual frame.
- The box model, display, position, and flexbox control where elements appear on the page.
- Media queries make layouts responsive by applying different CSS rules at different screen sizes.



Bibliography

- MDN <https://developer.mozilla.org/en-US/docs/Web/CSS>
- W3C schools <https://www.w3schools.com/css/default.asp>
- ACM Hack Materials tinyurl.com/hackschool1
- CSS tricks <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>
- Materials of the course *Computación en Red* - Universidad Politécnica de Madrid